

Master's thesis

When does Mamba2 beat the Transformer?

Energy per token, throughput and the crossover point of Mamba2-130m vs Pythia-160m and Pythia-70m on an NVIDIA RTX 3070

Sumon Kazi · Matriculation no. 23293505

Supervisor: Dr.-Ing. Ayesha Afzal · Winter semester 2026/27

Data Science
Faculty of Sciences

The short answer

~4K

tokens: Mamba2-130m overtakes the similar-size **Pythia-160m** at batch 1 ($\approx 6\text{--}8\text{K}$ at batch ≥ 2)

~46K

tokens: Mamba2 overtakes even the 4× cheaper **Pythia-70m**

14×

less energy per token than Pythia-160m at 256K tokens (batch 1)

Below the crossover the Transformers are faster and cheaper; above it, Mamba2's constant per-token cost wins on both throughput and energy.

Experimental setup

Component	Configuration
GPU	NVIDIA GeForce RTX 3070 · 8 GB · 220 W power limit · driver 610.57 (CUDA 13.3)
Host	FAU CS CIP pool (cip7b1) · Intel Core i7-12700 · 64 GB RAM · Debian Linux
Framework	Python 3.11 (conda) · PyTorch 2.12.1+cu130 · Transformers 5.12.1
Mamba kernels	mamba-ssm 2.2.6 + causal-conv1d 1.6.2, built from source (gcc 14.2, sm_86)
Energy API	NVML via nvidia-ml-py 13.610.43
Precision	float16, torch.inference_mode, random token ids

Models: same tokenizer, comparable size

Model	Architecture	Layers	d_model	Total params	Non-embedding
Mamba2-130m	State-space (SSD scan)	24	768	≈130M	≈90M
Pythia-160m	Transformer (softmax attention)	12	768	≈162M	≈85M
Pythia-70m	Transformer (softmax attention)	6	512	≈70M	≈19M

Embedding tables cost almost no compute, so **non-embedding** parameters are the fair size measure: Mamba2-130m vs Pythia-160m isolates the **architecture**; Pythia-70m vs Pythia-160m isolates **parameter count**.

Measurement methodology

1 · Idle baseline

5 s of GPU power at rest: 16.8 W

2 · Warm-up

≥ 3 iterations and ≥ 2 s; kernels compile, clocks ramp up

3 · Calibrate

Time one pass, then set iterations so each window lasts ≥ 4 s

4 · Measure

2–3 windows; NVML energy counter + 10 ms power polling

5 · Derive

$\text{tok/s} = B \cdot L \cdot \text{iters} \div t$
 $\text{mJ/token} = E \div \text{tokens}$

Single-stage inference: one forward pass over $B \times L$ tokens, logits for the last token only. GPU synchronisation brackets every window. The energy counter is the primary value; integrated polled power is the cross-check.

Why long windows: a forward pass of a small model can finish in < 100 ms, shorter than NVML's power update interval.

What theory predicts

Transformer

Per token and layer: linear layers $\approx 24 \cdot d^2$ FLOPs,
attention $\approx 2 \cdot L \cdot d$ FLOPs.

Attention dominates once $L \gtrsim 12 \cdot d$: $\approx 9K$ tokens for
Pythia-160m, $\approx 6K$ for Pythia-70m.

Total cost $\propto L^2$, so cost **per token** $\propto L$.

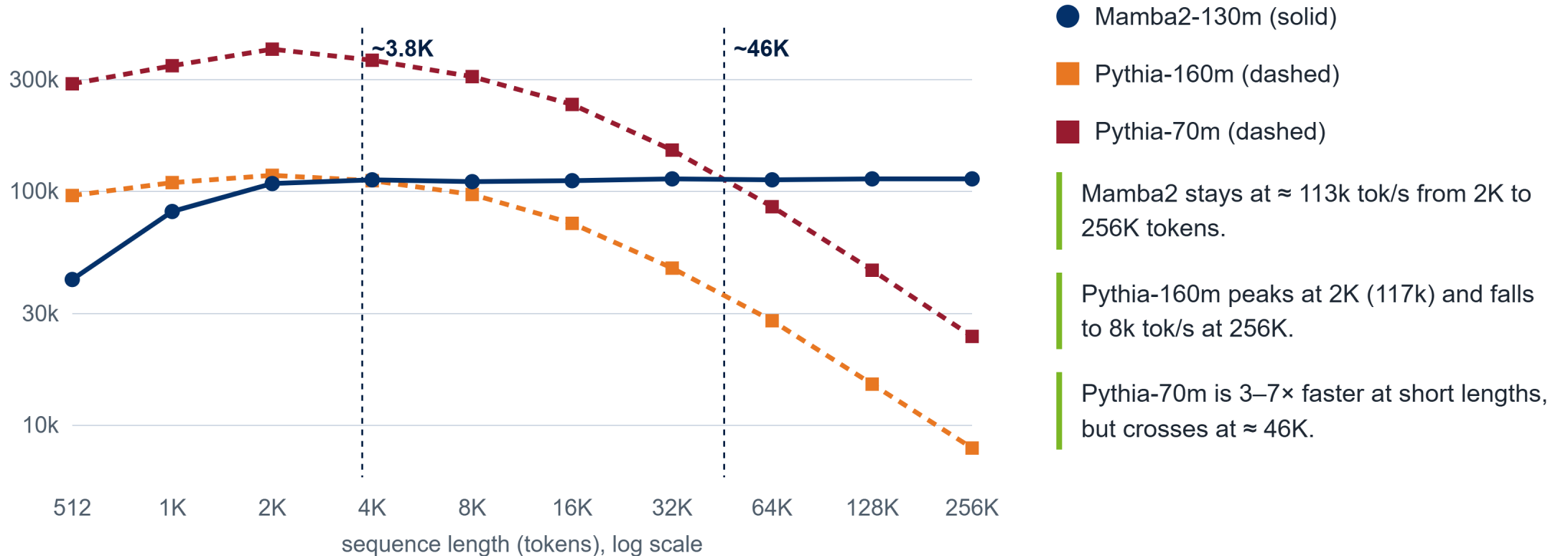
Mamba2

Selective state-space scan: fixed work per token,
independent of how many tokens came before.

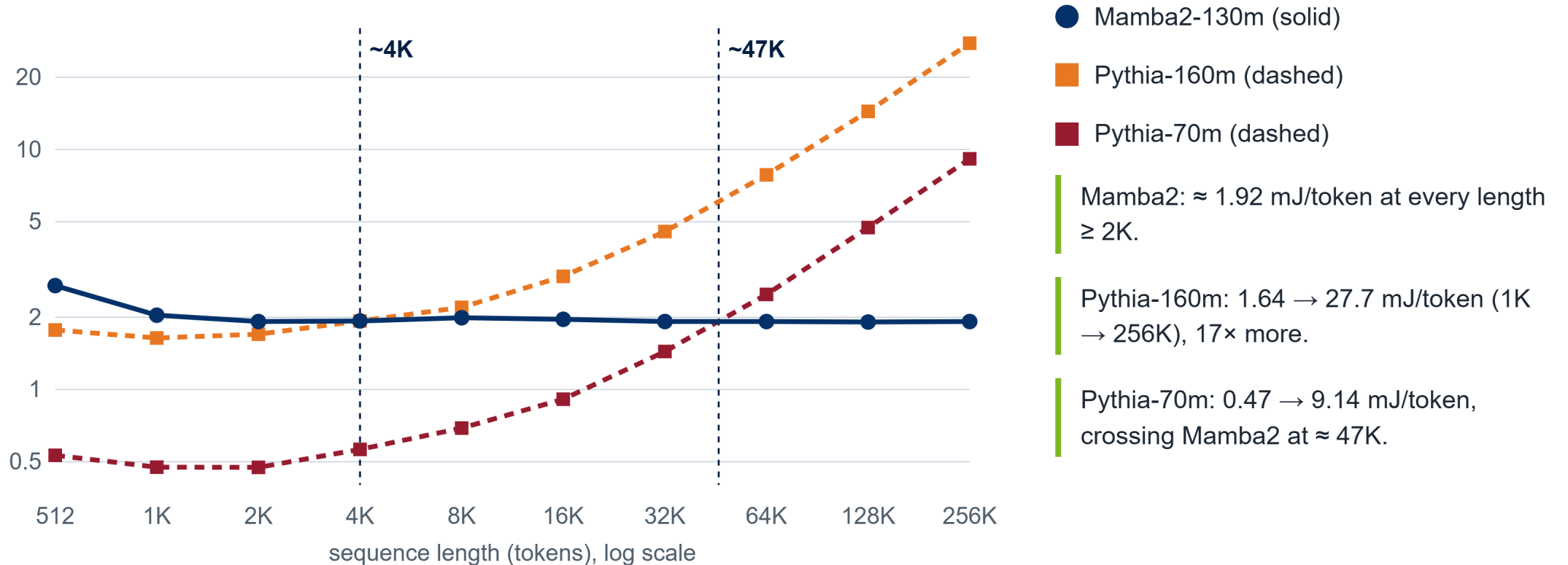
Total cost $\propto L$, so cost **per token is constant**.

Prediction: Transformers win at short lengths, Mamba2 wins at long lengths, and Transformer energy per token should approach $\times 2$ per doubling of L .

Throughput vs sequence length (batch 1)



Energy per token vs sequence length (batch 1)



Where the curves cross

Comparison	Throughput crossover	Energy crossover	Mamba2 advantage at the longest length
vs Pythia-160m, batch 1	≈ 3.8K tokens	≈ 4K tokens	14× (ratio 0.07 at 256K)
vs Pythia-160m, batch 2–16	≈ 6–8K tokens	≈ 6–8K tokens	1.5–7.7× (at the longest length that fits)
vs Pythia-70m, all batch sizes	≈ 46K tokens	≈ 47K tokens	≈ 5× (ratio 0.21 at 256K)

Crossovers are log-interpolated between measured points (factor-2 spacing), so they are estimates within one doubling. A 4× smaller Transformer pushes the crossover ≈ 12× further out, but does not remove it.

Does it match the theory? Mostly yes

Energy per token, growth per doubling of L	32K → 64K	64K → 128K	128K → 256K
Pythia-160m	× 1.72	× 1.84	× 1.92
Pythia-70m	× 1.73	× 1.90	× 1.93
Mamba2-130m	× 1.00	× 0.99	× 1.01

Quadratic regime confirmed

Transformer growth approaches the predicted $\times 2$ as attention takes over completely.

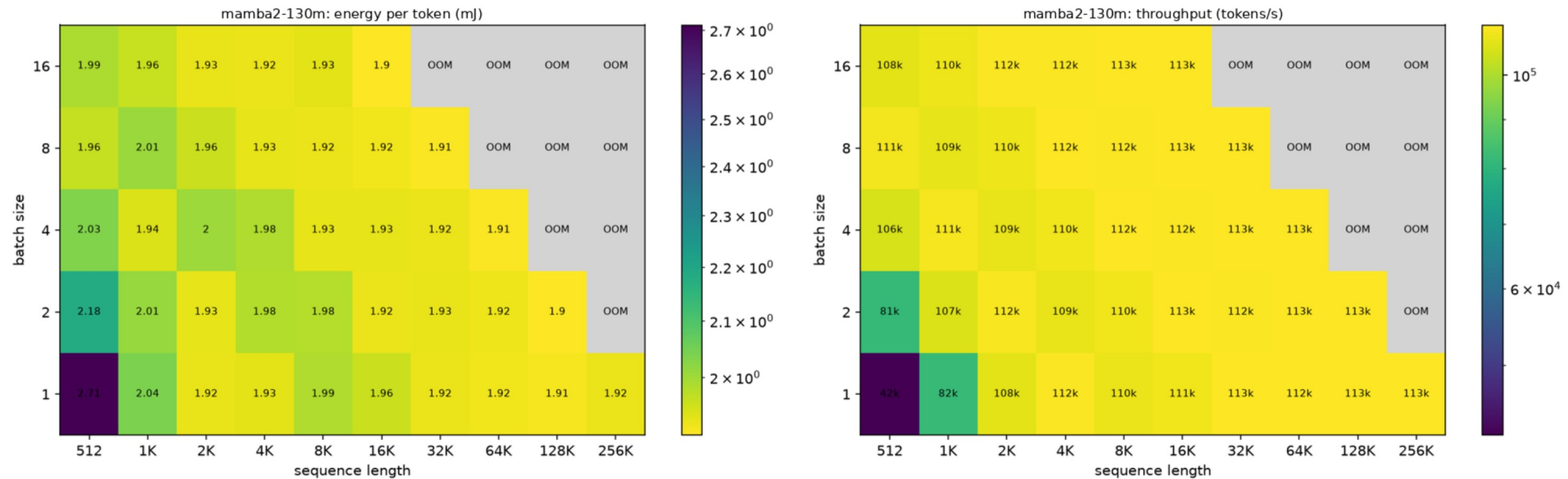
Linear regime confirmed

Mamba2's per-token cost is constant over two orders of magnitude of L.

Inflection as estimated

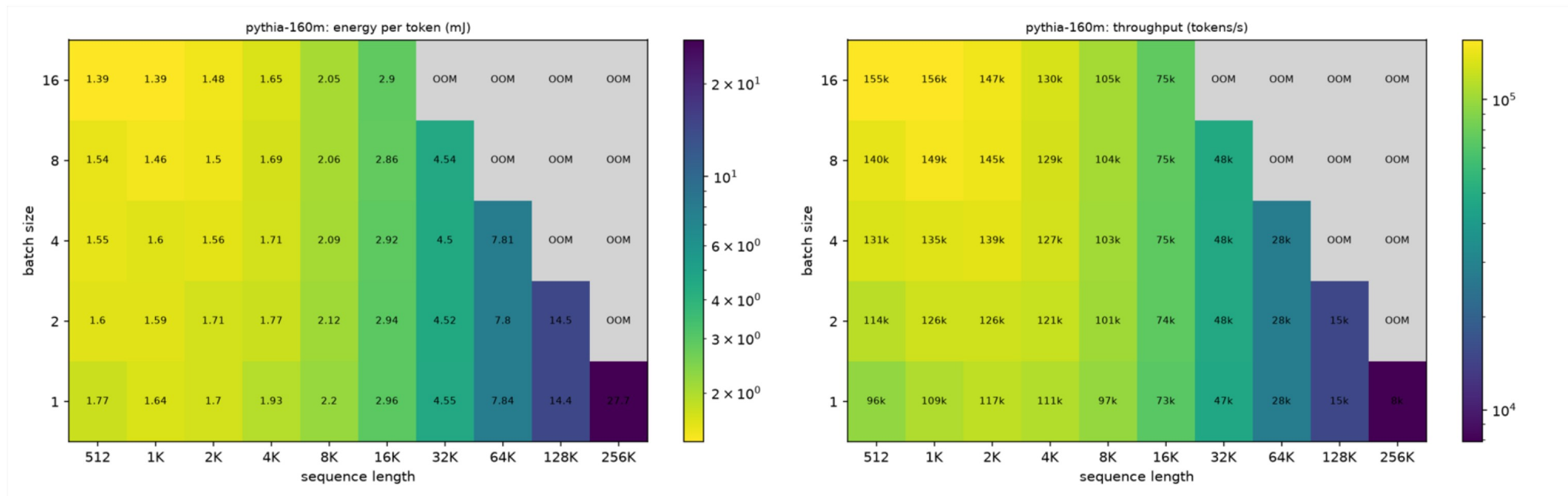
Transformer curves bend upward between 4K and 16K, consistent with $L \approx 12 \cdot d$.

Mamba2-130m: flat across the grid



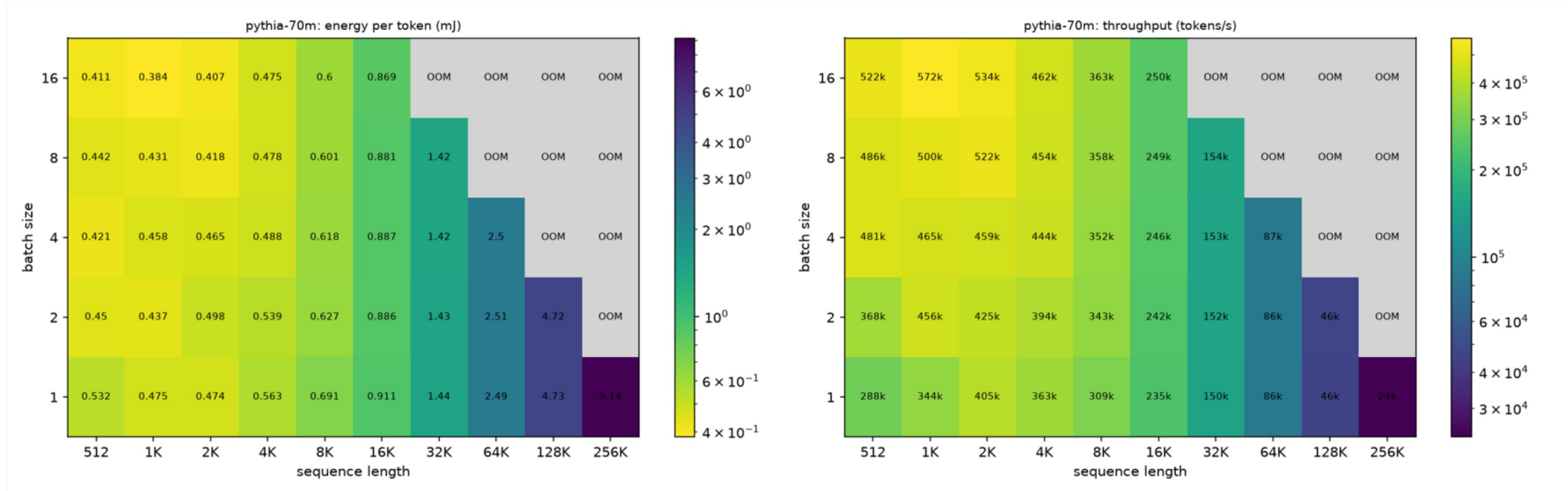
≈ 1.92 mJ/token and $\approx 113k$ tok/s wherever $B \cdot L \geq \sim 2\text{--}4K$ tokens; batching only helps at 512–1K. Grey = out of memory: the limit is the product $B \times L$.

Pythia-160m: cost rises with sequence length



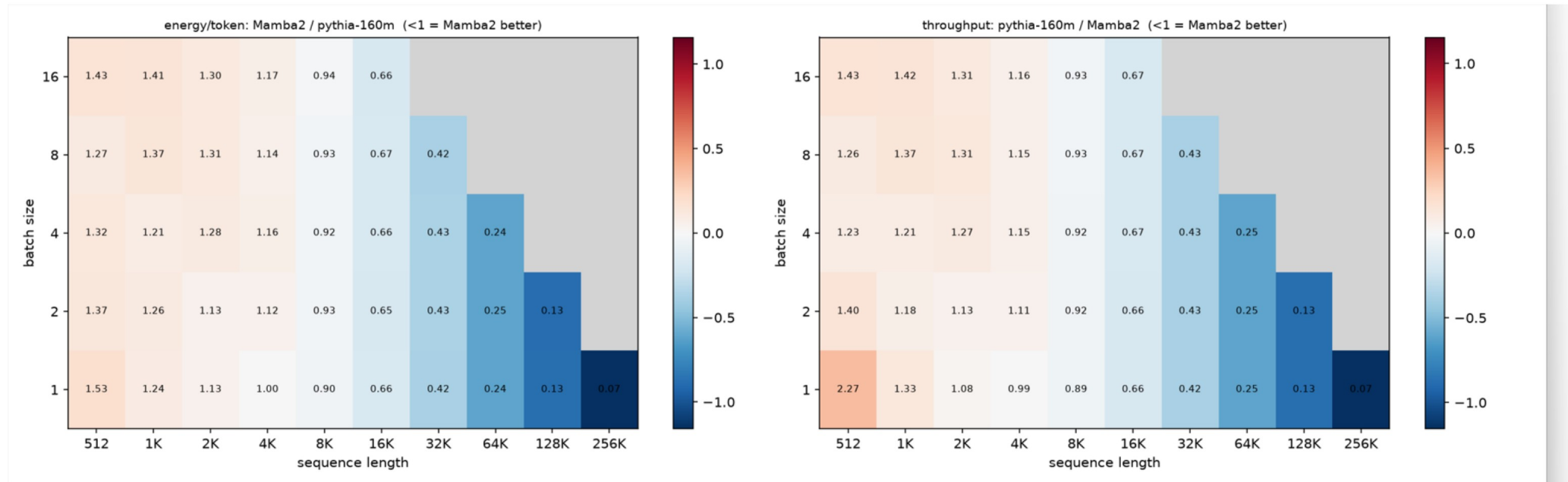
Columns change colour, rows barely do: sequence length drives the cost. Batching helps only at $\leq 4K$ (best: 1.39 mJ/token at $B = 8-16$, $L \leq 1K$).

Pythia-70m: very efficient short, quadratic long



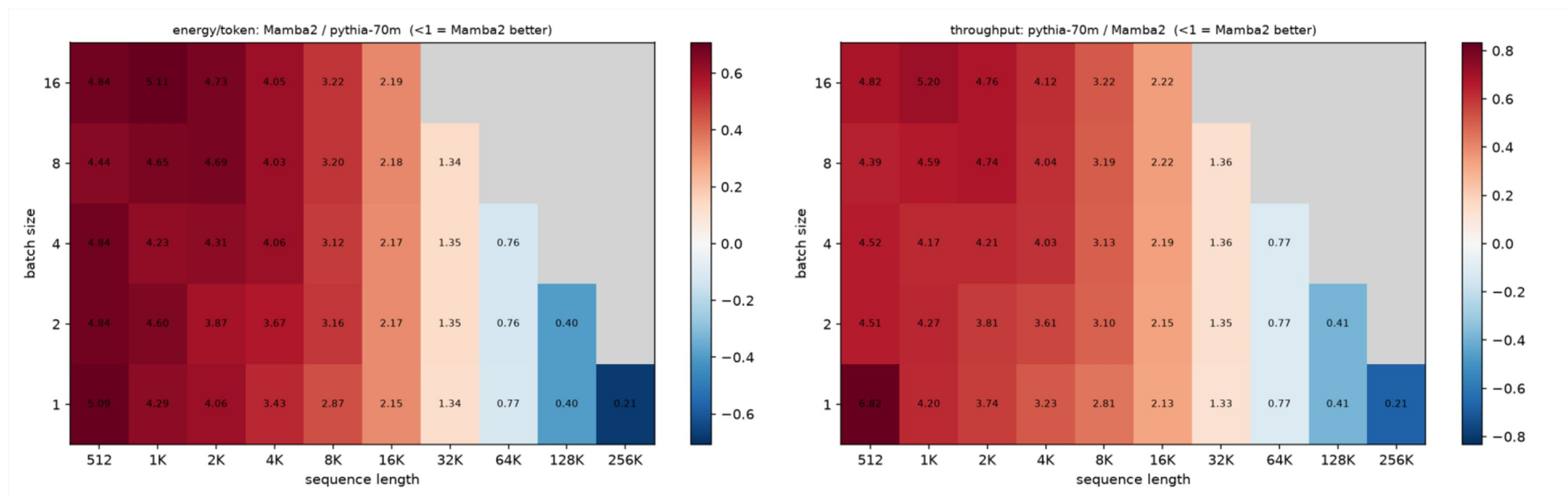
Best cell of the whole study: $B = 16, L = 1K \rightarrow 572k \text{ tok/s}$ at 0.384 mJ/token . At 256K tokens the cost rises to 9.14 mJ/token .

Mamba2 vs Pythia-160m: the intersection



Ratio < 1 (blue) = Mamba2 better. The white boundary is the crossover: $\approx 4K$ at batch 1, $\approx 6-8K$ at batch ≥ 2 ; at 256K Mamba2 needs only 7 % of the energy.

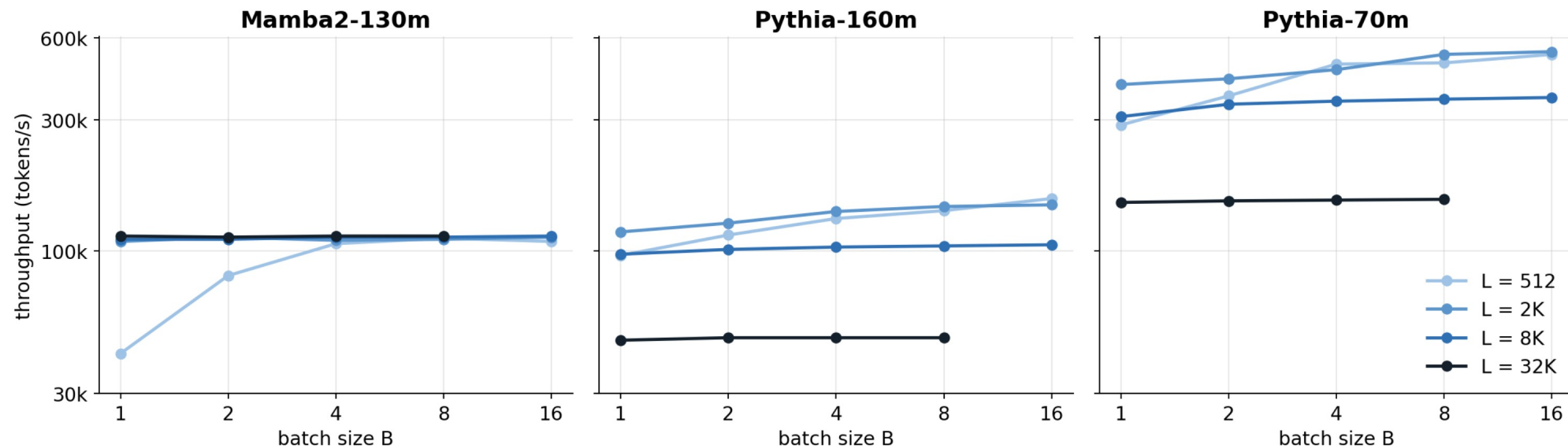
Mamba2 vs Pythia-70m: the intersection



Red up to 32K (Pythia-70m up to 5× cheaper), blue from 64K: the crossover lies between 32K and 64K for every batch size.

Throughput vs batch size

Figure 3. Throughput vs batch size (lines end where memory runs out)



Batching pays only for short sequences: at 512 tokens it raises throughput by about 60 % (Pythia-160m), 80 % (Pythia-70m) and 2.6× (Mamba2). At 8K and 32K all lines are flat. Lines end where memory runs out.

Batch size: helps only where the GPU is not yet full

Transformers

Pythia-160m at 512 tokens, B 1
→ 16: 96k → 155k tok/s, 1.77 →
1.39 mJ/token.

At $\geq 16K$: no gain; one long
sequence already saturates
the GPU.

Mamba2

Batching helps only at 512–1K
tokens: 42k → 108k tok/s.

Beyond $\approx 2\text{--}4K$ tokens per batch:
flat at $\approx 113k$ tok/s.

Memory budget

All three models run out of memory
at 512K tokens per batch; 256K fits.

The limit is the product $B \times L$, not
either factor alone.

Consequence: batching shifts the Pythia-160m crossover from $\approx 4K$ to $\approx 6\text{--}8K$ tokens.

Power is pinned at the limit

213–217 W

in almost every configuration, against a 220 W power limit

Constant power $\Rightarrow$ **energy per token $\approx$ power $\div$ throughput**; that is why energy and throughput heatmaps mirror each other.

Under-utilised GPU (B = 1, 512 tokens): 160–165 W. Mamba2's energy ratio there is 1.53 vs a throughput ratio of 2.27.

Idle floor of 16.8 W: an idle-ish GPU still costs energy, so low utilisation is never free.

Memory: theory not confirmed, and why

Measured

Peak memory at 32K tokens, batch 1: Mamba2 1164 MB · Pythia-160m 900 MB · Pythia-70m 531 MB.

All models hit out-of-memory at the same $B \times L$. No quadratic memory wall for the Transformers.

Why

PyTorch's default SDPA attention uses fused, memory-efficient kernels that never store the $L \times L$ matrix:
compute stays quadratic, memory becomes linear.

Mamba2's memory advantage shows in **decoding**: a constant state vs a growing KV cache, which a single forward pass does not measure.

Sweet spots

Model	Best configuration	Energy	Throughput
Mamba2-130m	Any batch with $B \cdot L \geq \sim 4K$ and $L \geq 2K$	≈ 1.90 mJ/token	$\approx 113k$ tok/s
Pythia-160m	$B = 8-16, L = 512-1K$	≈ 1.39 mJ/token	$\approx 155k$ tok/s
Pythia-70m	$B = 16, L = 1K$	0.384 mJ/token	572k tok/s

Transformers have a narrow sweet spot (short and batched). Mamba2 has a wide one: almost any configuration that fills the GPU.

Takeaways

1. Short contexts that can be batched: use a Transformer; a small one is cheapest of all.
2. Beyond $\approx 4\text{--}8\text{K}$ tokens (similar size) or $\approx 46\text{K}$ (a $4\times$ smaller Transformer), Mamba2 wins on both speed and energy.
3. Mamba2's cost per token is independent of length and batch size, so it is predictable.
4. On a power-limited GPU, energy per token is driven almost entirely by throughput.

Assumptions and limitations

Measurement scope

- Single forward pass (prefill-like); decoding and KV cache not measured
- Last-token logits only; random token inputs, no quality evaluation
- Pythia run beyond its 2048-token training context

Hardware and statistics

- NVML measures GPU power only (no CPU, RAM, PSU losses)
- Power-limited RTX 3070; results are GPU-specific; shared CIP machine
- 2–3 repeats per point; crossovers interpolated between doublings
- mamba-ssm 2.2.6 with a compatibility patch; SDPA attention

Thank you. Questions?

Code: `benchmark_energy.py` (Part 2) · `benchmark_sweep.py` (Part 3) · `plot_sweep.py` (figures)

Data: `sweep_b1.csv` · `sweep_grid.csv`